

49. Group Anagrams

Medium Topics Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

22.5K 429 1 1 1

403 Online

Code

Java Auto

```
1 import java.util.*;
2
3 class Solution {
4     public List<List<String>> groupAnagrams(String[] strs) {
5         Map<String, List<String>> map = new HashMap<>();
6
7         for (String str : strs) {
8             int[] count = new int[26];
9
10            // Count each character
11            for (char c : str.toCharArray()) {
12                count[c - 'a']++;
13            }
14
15            // Create a unique key from the frequency array
16            StringBuilder key = new StringBuilder();
17        }
```

Saved

Ln 31, Col 1

Testcase Test Result

Accepted Runtime: 1 ms

Case 1 Case 2 Case 3

Input

`strs = ["eat","tea","tan","ate","nat","bat"]`

Output

`[["bat"],["tan","nat"],["eat","tea","ate"]]`

Expected

`[["bat"],["nat","tan"],["ate","eat","tea"]]`

Contribute a testcase

Description Editorial Solutions Submissions

347. Top K Frequent Elements

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`
Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`
Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`
Output: `[1,2]`

Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than `O(n log n)`, where `n` is the array's size.

👍 19.8K 🗨️ 503 ☆ 📄 ⓘ

349 Online

</> Code Java ⏏️ Auto ⋮ 📄 🔍 ↺ ↻ ↗

```
1 import java.util.*;
2
3 class Solution {
4     public int[] topKFrequent(int[] nums, int k) {
5
6         // Count frequencies
7         Map<Integer, Integer> frequency = new HashMap<>();
8
9         for (int num : nums) {
10             frequency.put(num, frequency.getOrDefault(num, 0) + 1);
11         }
12
13         // Min-heap based on frequency
14         PriorityQueue<Integer> minHeap = new PriorityQueue<>()
15             {
16                 (a, b) -> frequency.get(a) - frequency.get(b)
17             };
18     }
```

Saved Ln 37, Col 1

✔️ Testcase ➤ Test Result

Accepted Runtime: 1 ms

✔️ Case 1 ✔️ Case 2 ✔️ Case 3

Input

nums =
[1,1,1,2,2,3]

k =
2

Output

[1,2]

Expected

[1,2]